Multimedia Extensions for a 550-MHz RISC Microprocessor

David A. Carlson, Ruben W. Castelino, and Robert O. Mueller

***Abstract*—This paper describes circuits used to implement the motion video instructions (MVI) in the 550-MHz Alpha 21 164PC Microprocessor. The chip is fabricated in a 0.35-μm CMOS process and is the first implementation of the MVI instruction set in the Alpha architecture. The MVI instruction set, coupled with the high performance of the 550-MHz processor, delivers 30 frames/s digital video disk (DVD) playback with stereo-quality audio, enables video teleconferencing at 30 frames/s, and significantly improves the performance of MPEG encode algorithms.**

***Index Terms*—Adders, alpha, CMOS digital integrated circuits, DVD, H.263, microprocessors, MPEG, multimedia computing.**

I. Introduction

THE 21 164PC is a 1.38 cm^2 microprocessor fabricated in a four-layer metal, 0.35-μm CMOS process with 3.5M transistors. This superscalar microprocessor can issue two integer instructions and two floating point instructions every CPU cycle. The chip contains a 16-kB, direct-mapped, virtual instruction cache and an 8-kB dual-ported physical data cache. An on-chip cache controller supports an external cache using pipelined burst SRAM's to deliver 3.2 Gbyte/s data bandwidth [4]. The estimated performance is 12 SPECint95 and 16 SPECfp95. Parts of the design were leveraged from the 21 164 microprocessor [2], [3]. Fig. 1 shows a photomicrograph of the chip without the upper two metal layers. Table I summarizes the chip specifications.

This microprocessor is the first implementation of the motion video instruction (MVI) extensions to the Alpha architecture [6]. MVI greatly increases the computational capability of this microprocessor to execute multimedia applications. In particular, MVI boosts the performance of motion estimation algorithms and enables H.261 or H.263 teleconferencing at 30 frames/s and digital video disk (DVD) playback at 30 frames/s with stereo quality audio.

The MVI extensions add three single-instruction, multiple-data (SIMD) class instructions to the Alpha architecture for a total of 13 new instructions. These new instructions are summarized in Table II. The vectorized MAX/MIN instructions choose the maximum or minimum values of pairs of eight 8-b or four 16-b words in a single instruction. The PACK/UNPACK instructions are used to compress or decompress byte and word data for efficient data manipulation. For example, the UNPKBL instruction takes two adjacent bytes and right shifts with a zero extend so the original data now occupies two adjacent 32-b words. The UNPKBW instruction is similar in that it takes four bytes and expands them to adjacent 16-b words. The PKLB and PKWB do the inverse of the UNPACK instructions. The combination of the MAX/MIN and PACK/UNPACK instructions enable this processor to

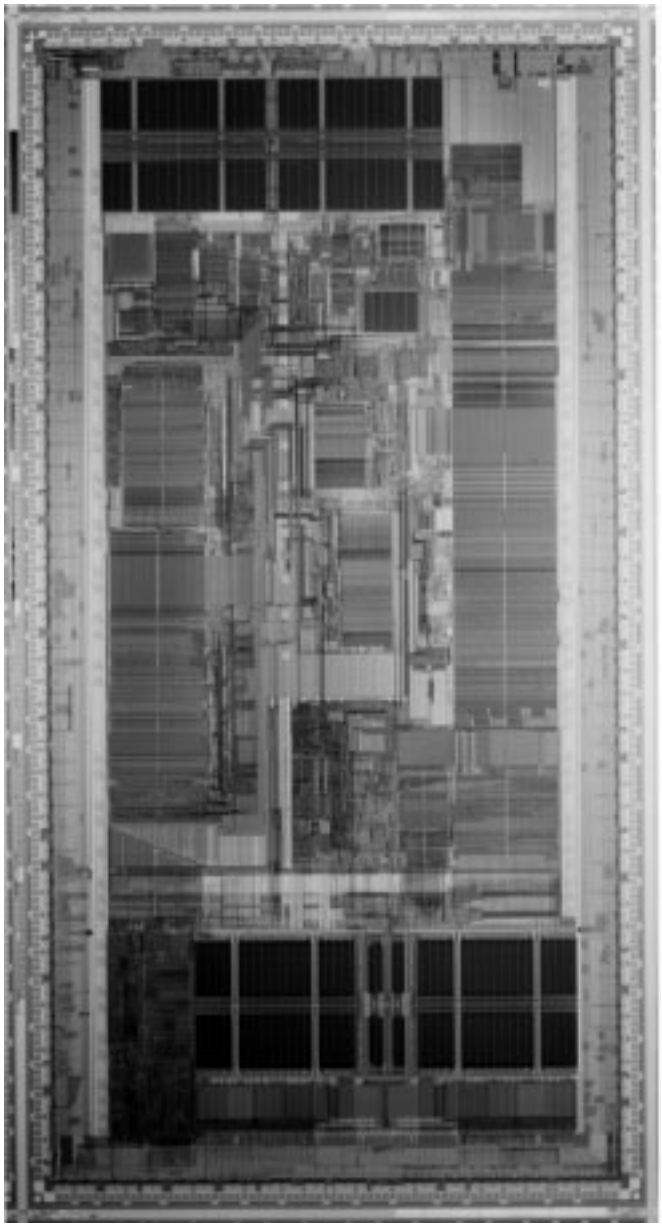


Fig. 1. Chip photomicrograph.

Manuscript received April 10, 1997; revised July 1, 1997.
The authors are with Digital Equipment Corp., Hudson, MA 01749 USA.
Publisher Item Identifier S 0018-9200(97)08027-X.

0018–9200/97$10.00 © 1997 IEEE

Authorized licensed use limited to: University of Sydney. Downloaded on November 03,2026 at 22:54:07 UTC from IEEE Xplore. Restrictions apply.

TABLE I
CHIP IMPLEMENTATION SPECIFICS

Transistors	3.5 Million
Die Sixe	8.6 mm × 16.1 mm = 1.38 cm^2
Process	0.35-μm N-Well CMOS four levels of Al interconnect
Frequency	100 MHz to 550 MHz
Power	30 W @ 500 MHz, 2.5 V VDDi

TABLE II
MVI INSTRUCTIONS

Instruction Class	Instructions	Description
MAX/MIN	MAXSB8	Pick MAX of each byte (signed)
	MAXSW4	Pick MAX of each 16 bit word (signed)
	MAXUB8	Pick MAX of each byte (unsigned)
	MASUW4	Pick MAX of each word (unsigned)
	MINSB8	Pick MIN of each byte (signed)
	MINSW4	Pick MIN of each word (signed)
	MINUB8	Pick MIN of each byte (unsigned)
	MINUW4	Pick MIN of each word (unsigned)
PACK/UNPACK	PKLB	Pack Long Words (32 b) to bytes
	PKWB	Pack Words (16 b) to bytes
	UNPKBL	Unpack bytes to Long Words
	UNPKBW	Unpack bytes to words
Pixel Error	PERR	Sum of absolute values of byte differences

```
Cycles
1 cmpbge   r0,r1,r23      Move larger bytes into r1,
  xor      r0,r1,r24      and smaller bytes into r0
2 zapnot   r24,r23,r23    to get positive result for
3 xor      r0,r23,r0      subtract
  xor      r1,r23,r1
4 subq     r1,r0,r0       Subtract pixels.
5 zap      r0,#85,r1
6 zapnot   r0,#85,r0      Expand byte results into
7 srl      r1,#8,r1       words,align, and then sum.
  addq     r0,r10,r10
8 addq     r1,r11,r11
◆8 Pixels processed in 8 cycles
```

Fig. 2. Pre-MVI macrocode for PERR instruction.

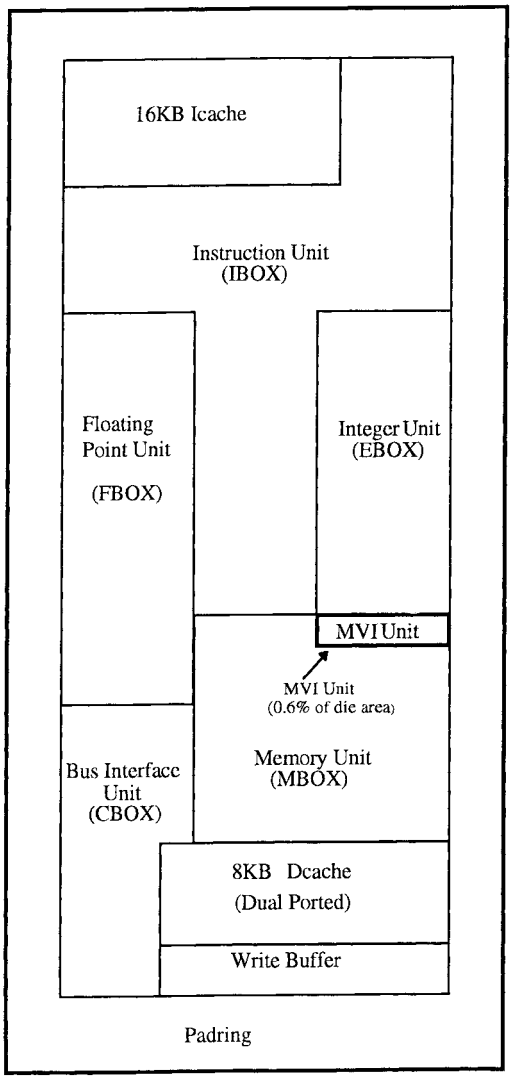


Fig. 3. Chip floorplan.

efficiently process byte and 16-b word-length pixel data for multimedia applications. The pixel error (PERR) instruction computes the sum of the absolute differences of pairs of 8 bytes. This instruction greatly improves the performance of many motion estimation algorithms including MPEG-I and MPEG-II.

In many MPEG encoding algorithms, 70% of the cycles are spent doing motion estimation [1]. Generally, 80% of the motion estimation code is pixel error calculation. This means that 56 out of every 100 cycles of MPEG encoding are spent calculating pixel errors (assuming that the data is not bandwidth limited). Fig. 2 shows a code example of the inner loop of a pixel error calculation which can be replaced by a single, pipelined PERR instruction, thus, improving the execution time of this portion by a factor of eight. For the entire MPEG encode routine, this brings us from the baseline of 100 cycles down to just 51 cycles [100 − 56 + (56/8)]. This is almost a 2× speedup due solely to the PERR instruction.

In addition to this speedup, the use of the PERR instruction in the inner loop allows additional unfolding of other pieces of code surrounding the PERR loop allowing for better use of the dual issue and bandwidth capability of the processor. This additional unfolding can further improve the pixel error calculation by a factor of six, resulting in a 2.2× speedup in the overall application.

The remaining MVI instructions enhance the ability of many video compression and decompression algorithms to handle byte and word data more efficiently, resulting in further performance improvements in MPEG encoding. The PACK instruction can save many cycles preparing data for the pixel error and other pixel-based calculations as well as allowing for more efficient use of valuable data storage resources. The MAX/MIN instructions save time when clipping additive pixel data back to the proper values for a given calculation.

As seen in Fig. 3, the fully pipelined MVI logic accounts for roughly 0.6% of the die area and operates at the same frequency as the rest of the processor. It was implemented as part of the integer datapath which allows the MVI logic to receive and supply data to and from the other integer functional units via the existing bypass circuitry. Installing the MVI circuitry in the integer unit, as opposed to the

Authorized licensed use limited to: University of Sydney. Downloaded on November 03,2026 at 22:54:07 UTC from IEEE Xplore. Restrictions apply.

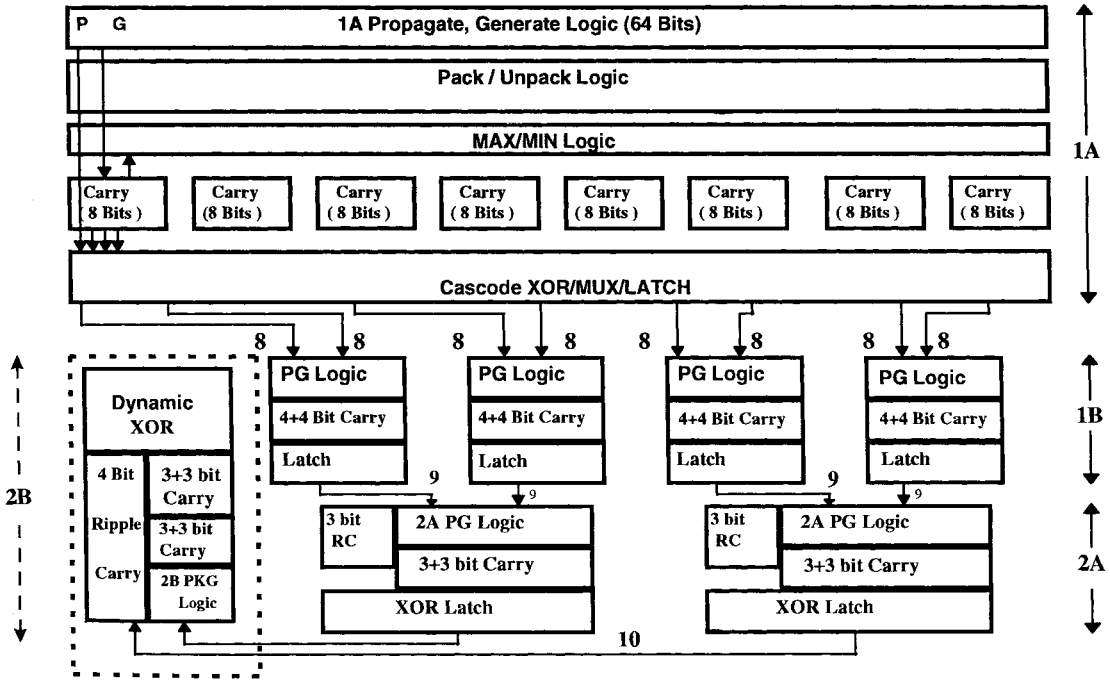


Fig. 4. MVI unit floorplan.

floating point unit [5], [7], prevented the costly duplication of existing integer functions such as logical operations and shifts. The issue bandwidth of two integer instructions per cycle, combined with 31 general purpose registers and low instruction latencies, was sufficient to fully utilize MVI's capabilities.

II. Design Constraints

Implementing 13 new instructions on a leveraged design created several challenging design constraints. The first is that the issue unit could only support zero-, one-, and two-cycle latencies. This posed an implementation challenge for the PERR instruction. Fundamentally, PERR requires six distinct operations: Compute an 8-b difference, calculate an absolute value, perform an 8-b add, perform a 9-b add, perform a 10-b add, and forward the result to other integer units. From an implementation perspective, a three-cycle instruction would have been simpler and smaller but this would have required significant rework of the issue unit. Because of this, PERR was implemented as a two-cycle instruction requiring an aggressive circuit implementation.

Another design constraint revolved around the bypass lines whose length and loading could not be increased. The bypass lines implement a dynamic distributed MUX for the operand data which is forwarded between the various integer functional units. Increasing the length of these lines would have aggravated capacitive coupling and increased the interconnect capacitance. Adding another driver for the MVI unit would have increased the delay of a single phase (1 ns) critical path by approximately 5%. Because of these reasons, the bypass lines stop at the top of the MVI unit and an additional driver was not used.

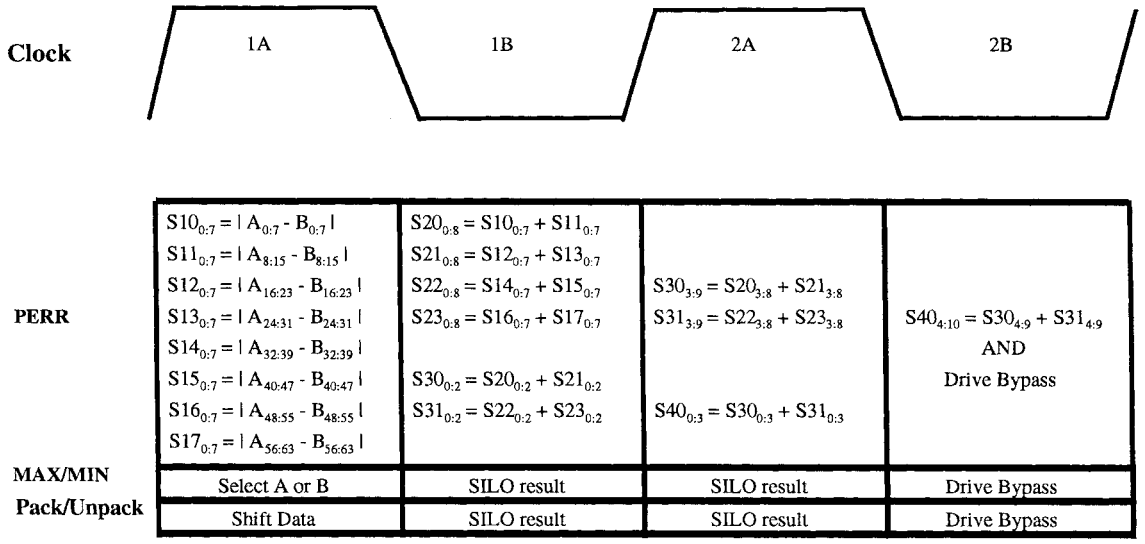
III. Floorplan

A single-wire two-phase clocking style was used throughout this design [2]. The 1A, 1B, 2A, 2B notation in Fig. 4 refers to the high and low phase of this clock. Fig. 4 also shows the MVI floorplan with the approximate dimensions of 1400 $\times$ 600 μm. The MVI unit was placed at the bottom of the integer unit above the memory unit. All of the source data from the register file or other integer functional units arrives at the top of the MVI unit on two dynamic differential bypass busses. A dynamic differential fill bus runs right over the MVI unit returning data for a load to the integer unit. Because of the two-cycle latency chosen for MVI, it is architecturally impossible for a load and an MVI result to arrive at the same time. This allows the MVI result to be muxed directly on the dynamic fill bus. The integer unit logic mimics a load fill operation to insert the MVI result back into the pipeline. This optimization prevented the addition of another bypass driver on the bypass lines saving approximately 85 μm in height and eliminating significant loading on a critical path.

IV. Implementation

The PACK/UNPACK instructions were implemented using three 8-b dynamic bidirectional busses. Located under the PACK/UNPACK logic is the MAX/MIN logic which is quite small due to the fact that the instructions take advantage of logic used for pixel error. For example, the PERR logic calculates whether $A < B$ or $A \leq B$ for each 8-b pair (A, B). Another signal needed for signed MAX/MIN was the XOR of the sign bits. This happens to be a propagate signal already present in the PERR logic. Both the MAX/MIN and PACK/UNPACK results are available after one CPU cycle but they are pipelined an additional cycle. This allows all of the

Authorized licensed use limited to: University of Sydney. Downloaded on November 03,2026 at 22:54:07 UTC from IEEE Xplore. Restrictions apply.



PERR	$S10_{0:7} = \lvert A_{0:7} - B_{0:7} \rvert$ $S11_{0:7} = \lvert A_{8:15} - B_{8:15} \rvert$ $S12_{0:7} = \lvert A_{16:23} - B_{16:23} \rvert$ $S13_{0:7} = \lvert A_{24:31} - B_{24:31} \rvert$ $S14_{0:7} = \lvert A_{32:39} - B_{32:39} \rvert$ $S15_{0:7} = \lvert A_{40:47} - B_{40:47} \rvert$ $S16_{0:7} = \lvert A_{48:55} - B_{48:55} \rvert$ $S17_{0:7} = \lvert A_{56:63} - B_{56:63} \rvert$	$S20_{0:8} = S10_{0:7} + S11_{0:7}$ $S21_{0:8} = S12_{0:7} + S13_{0:7}$ $S22_{0:8} = S14_{0:7} + S15_{0:7}$ $S23_{0:8} = S16_{0:7} + S17_{0:7}$ $S30_{0:2} = S20_{0:2} + S21_{0:2}$ $S31_{0:2} = S22_{0:2} + S23_{0:2}$	$S30_{3:9} = S20_{3:8} + S21_{3:8}$ $S31_{3:9} = S22_{3:8} + S23_{3:8}$ $S40_{0:3} = S30_{0:3} + S31_{0:3}$	$S40_{4:10} = S30_{4:9} + S31_{4:9}$ AND Drive Bypass
MAX/MIN	Select A or B	SILO result	SILO result	Drive Bypass
Pack/Unpack	Shift Data	SILO result	SILO result	Drive Bypass

Fig. 5. MVI instruction timing table.

MVI instructions to have a common two-cycle latency. This tradeoff has a negligible performance impact due to the vector nature of MVI code but did reduce the design complexity and area required.

Pixel error is a much more challenging instruction as can be seen from the timing diagram in Fig. 5. Note that every phase requires at least an 8-b add. The 1A clock phase requires an absolute value function to be performed on an 8-b difference. The absolute value is tightly coupled to the difference operation resulting in an efficient circuit block. Fig. 6 shows an abstraction of this logic. The carry blocks at the top of the figure utilize carry select to yield two carry chains that are effectively 8 b wide. The output of the carry chains is

$$C0 = (A + \overline{B} + 1) \oplus P$$
$$C1 = (A + \overline{B} + 0) \oplus P.$$

The final answer of $A-B$ is generated by the XOR of $C0$ and P

$$(A + \overline{B} + 1) \oplus P \oplus P \Rightarrow A + \overline{B} + 1 \Rightarrow A - B.$$

The final answer of $B - A$ is generated by the XNOR of $C1$ and P

$$(A+\overline{B}+0) \oplus P \oplus \overline{P} \Rightarrow \overline{(A + \overline{B} + 0)} \Rightarrow \overline{A}+B+1 \Rightarrow B-A.$$

The final carry-out of the addition of $A - B$ will be a one if $A > B$. This carry-out is then used to select the positive answer yielding the absolute value function. Instead of using a carry select technique to determine this signal, a three-input complex gate was used for speed reasons.

Fig. 7 shows the detailed circuit making up the carry chain subblock from the previous section. The high level abstraction hides many optimizations available at the circuit level. For example, there are four leftmost carry chains instead of the logically required two. The seemingly redundant two carry chains (CC1, CC3), without the intermediate output inverters, are used to drive carry select muxes. Eliminating the inverter loads from these carry chains greatly improves the critical path

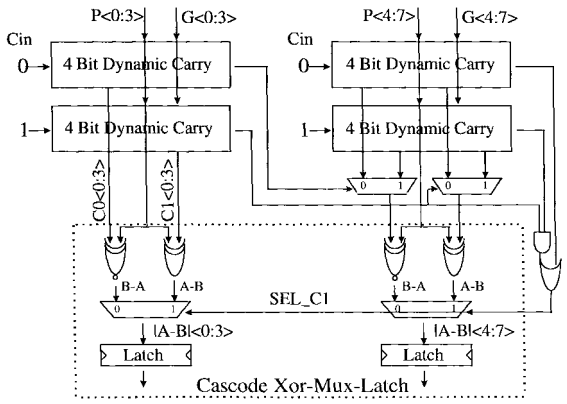


Fig. 6. Eight-bit absolute difference logic.

but has no area impact because the height of this cell is set by the rightmost logic.

The rightmost logic implements two 4-b carry chains plus the carry select muxes. The top chain (CC5) is similar to the redundant chains on the left in that its sole purpose is to quickly evaluate the complex gate. The mux used for the carry select is done efficiently using a single transistor per bit and another carry chain (CC7) with a propagate term connected to VSS. It is possible to do carry select this way because a carry-in can only introduce a low value on the dynamic logic. This allows chain CC7 to precompute the value for a carry-in of one. If the logic to the left determines the carry-in really is a one, then chain CC7 discharges chains CC6 and or CC8. This same approach could have been used to determine the final carry-out of the adder needed for selecting the positive difference. In fact, this was done for the $A_LESSEQUAL_B$ signal which was not time critical. However, the SEL_C0 signal was needed sooner so a faster method was employed using the complex gate located in the bottom right corner. By using this complex gate, the delay of a dynamic stage was eliminated producing the SEL_C0 and SEL_C1 signal quickly.

The last piece of the absolute value function is based on the cascode circuit shown in Fig. 8. This circuit implements a

Authorized licensed use limited to: University of Sydney. Downloaded on November 03,2026 at 22:54:07 UTC from IEEE Xplore. Restrictions apply.

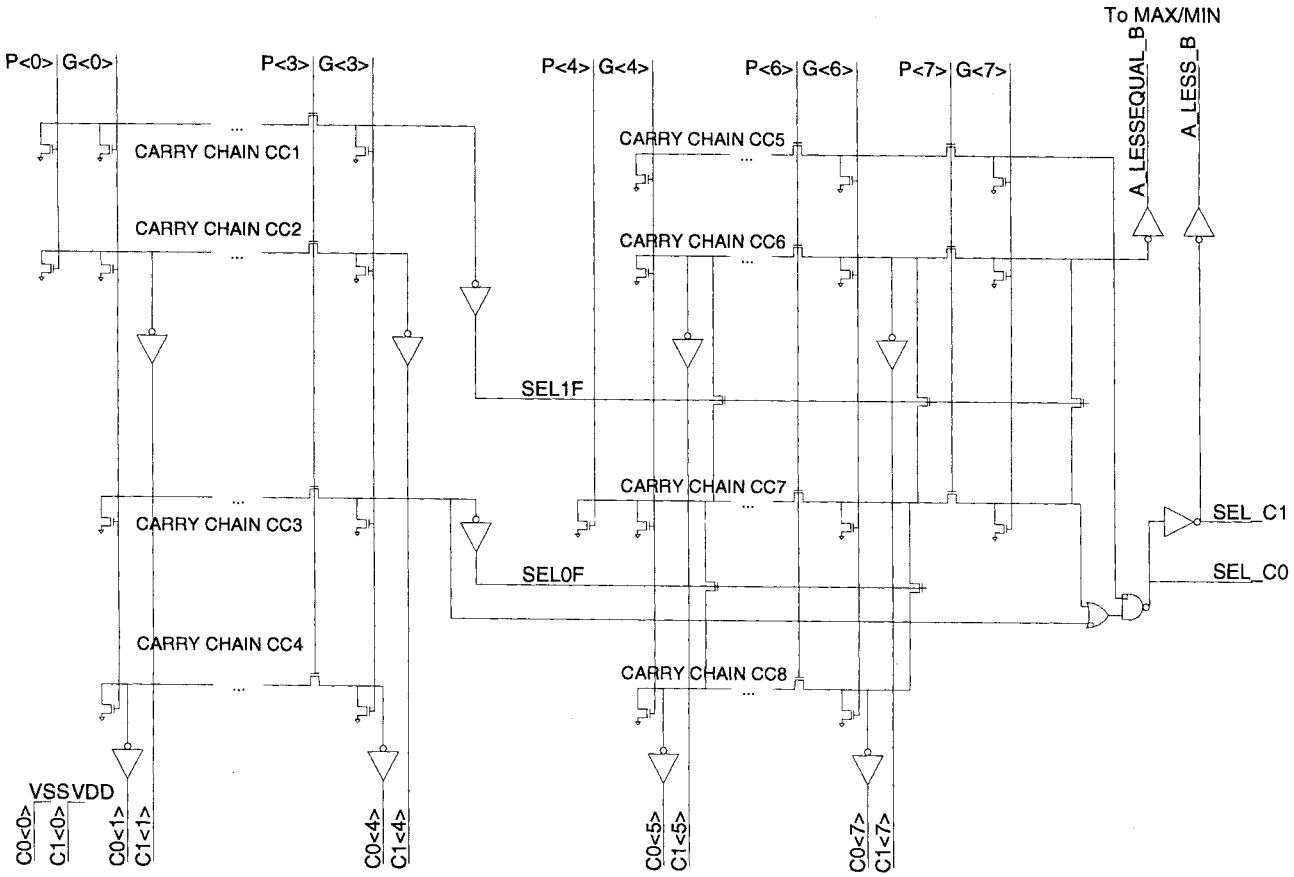


Fig. 7. Absolute difference carry chains.

two-input XOR and multiplexer. All of the carry inputs are locally inverted to achieve both the true and complement signals necessary for cascode logic. The propagate term goes into the bottom of the stack and is set up well before the carry terms arrive. This cascode circuit relies on the fact that the $C0$ and $C1$ inputs are not independent. It was noticed that the $C0$ input can never be a one without the $C1$ input also being a one. This observation allowed for a simplification of the cascode logic. Differential routing of the cascode output to the next stage was not used. This incurred the penalty of an inverter at the next stage but eliminated half of the horizontal metal tracks needed for routing.

The 1B and 2A stages of PERR are very similar to the first stage, in that dynamic logic is used to create the propagate and generate signals. This is followed by dynamic carry chains and then a cascode XOR with latch. The logic needed for absolute value is, of course, eliminated. For the 1B clock phase, where clock is low, P-domino and P-cascode were not used as expected. Instead, the same N-domino and N-cascode structures were used with an inverted clock. Using N devices made a smaller and faster design at the expense of adding a potential race due to the skew from the inversion of clock. This race was analyzed using SPICE and easily managed.

The greatest challenge for the PERR circuit implementation was in the final 2B phase where a 10-b add is required. Because the result could be consumed the next clock phase by other integer functional units, the result of this add must be forwarded to the other Ebox units in the same phase as the add. This only leaves half of the phase, or approximately 500 ps, to do the add. This is insufficient time for using domino logic to compute the PG's, propagate two 5-b carry chains, and use a cascode XOR like the rest of the PERR logic. Instead, this final add was done differently. The propagate and generate signals were precomputed in phase 2A and fed into a clocked NOR gate. Two 3-b carry chains were used resulting in 6 b of the add being accomplished. The XOR stage was done with dynamic logic that requires true and complement versions of all signals. This doubled the PG logic as well as doubling the carry chains to compute the carry complements. Reducing the carry chains to 3 b instead of 5 b significantly improved the delay of the carry chains. The remaining 4 b were done in phase 2A with a static ripple carry adder. The time to add 4 b with a ripple carry adder in phase 2A was made possible by adding 3 b of the phase 2A add in 1B. The placement of the final add in the floorplan was made so that its more complex logic did not set the height of the MVI unit.

Fig. 9 shows the SPICE waveforms for signals in the final PERR stage. This simulation was run at worst case conditions of 1.8 V with a cycle time of 2.4 ns. The figure shows the carry-in from the ripple carry adders (cin) arriving just before the falling clock edge. The propagate signal (p_s4) going to the 3-b carry chains rises 0.14 ns after the clock edge. The output of the most significant bit of the carry chain (c0_s4) rises at 0.3 ns driving the pull down of a dynamic XOR gate. This XOR gate causes the load fill bus (result) to fall at 0.7 ns after

Authorized licensed use limited to: University of Sydney. Downloaded on November 03,2026 at 22:54:07 UTC from IEEE Xplore. Restrictions apply.

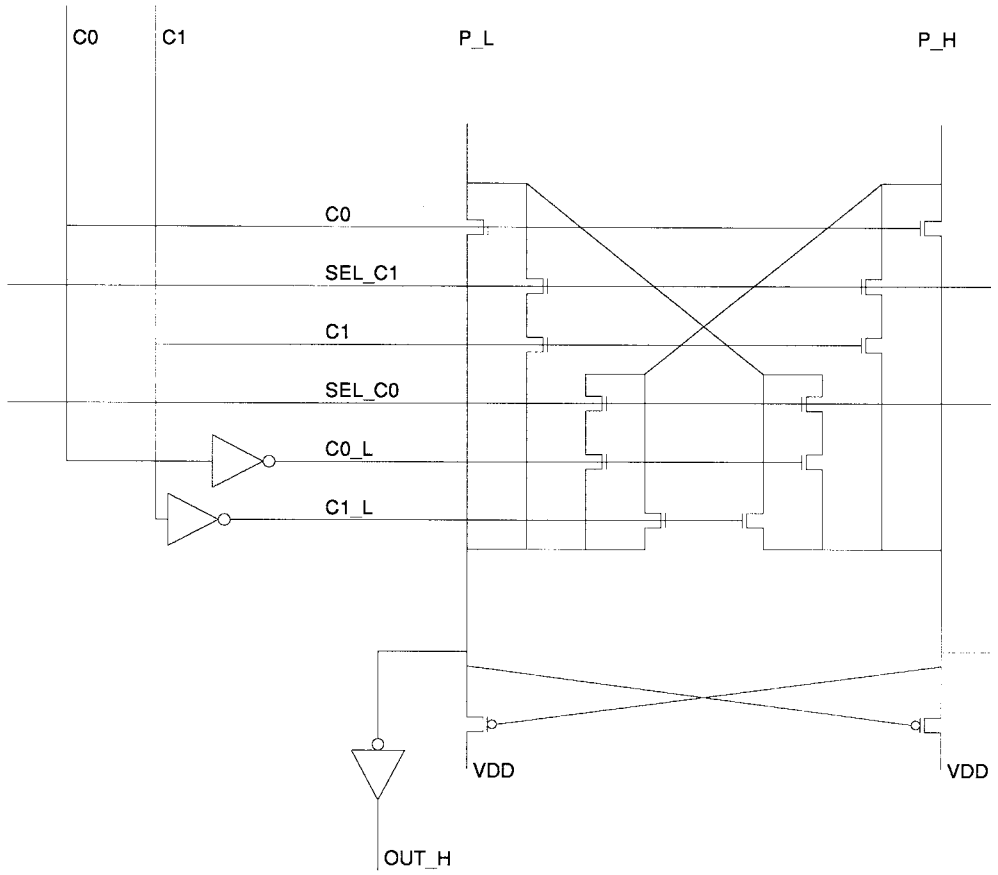


Fig. 8. One-bit slice of cascode XOR/Mux.

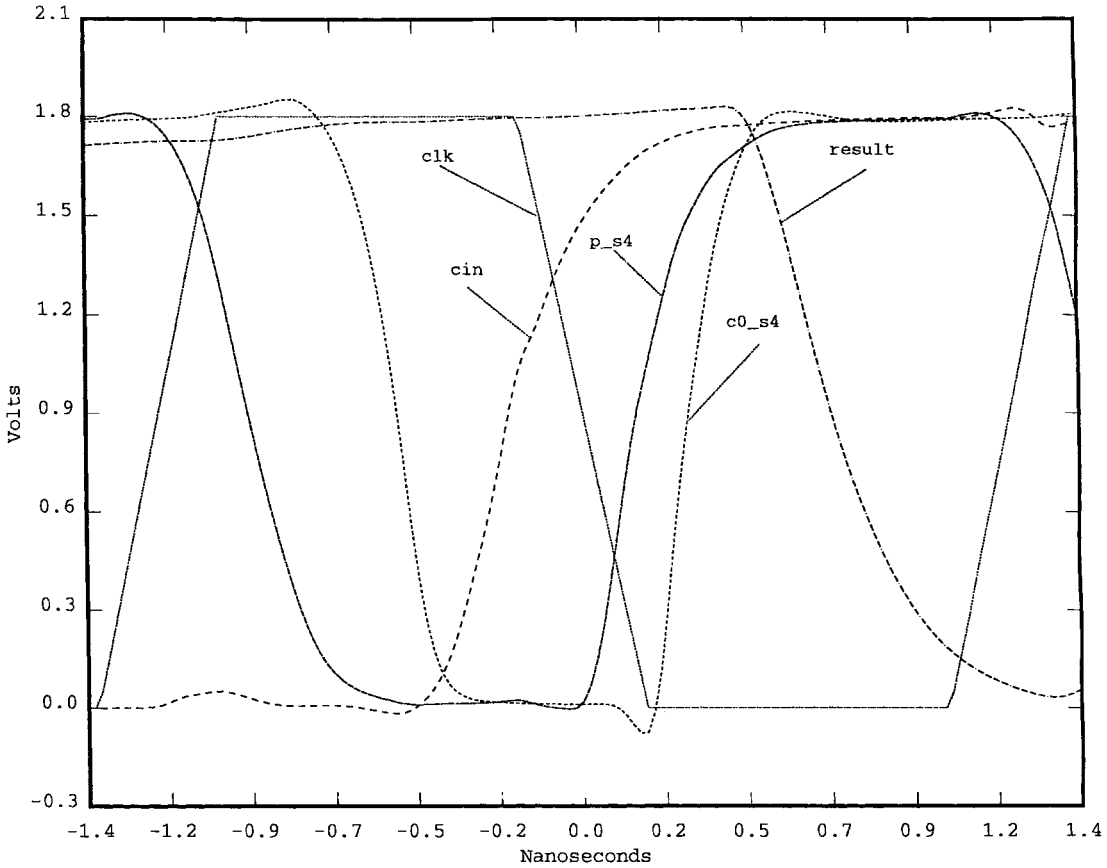


Fig. 9. SPICE results of final 2B PERR stage.

Authorized licensed use limited to: University of Sydney. Downloaded on November 03,2026 at 22:54:07 UTC from IEEE Xplore. Restrictions apply.

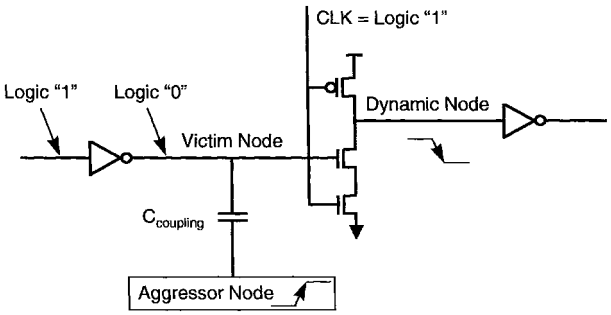


Fig. 10. Dynamic logic coupling diagram.

the clock edge. This leaves approximately 0.5 ns available to forward the data to the other integer functional units.

V. Capacitive Coupling

Although the methods and tool suite used for electrical verification are the same as those for the 21 164 microprocessor [3], it is instructive to discuss one area of importance: capacitive coupling. Much of the speed of this Alpha microprocessor can be attributed to the widespread use of dynamic logic throughout the design. The MVI circuits are a good example of this. Coupling is a particular concern with dynamic logic. Fig. 10 shows the problematic case. The victim wire is feeding an N-pulldown to dynamic logic. In the evaluate phase, the victim has a low value but is being coupled high by the aggressor node. If the intersignal capacitance of the victim is a significant proportion of the victim's total capacitance, then the victim's voltage will rise enough to erroneously discharge the dynamic logic. Coupling problems were circumvented in several ways. First, in the design phase, signals that were anticipated to be coupling problems were routed with greater than minimum spacing from aggressor signals. In some cases, where area permitted, a minimum spaced shield tied to VDD or VSS was placed adjacent to the victim wire. After the layout was complete, an in-house capacitance extraction tool was run to accurately determine intersignal capacitance between signals on the same and different metal layers. After this, a CAD tool was used to determine failures by incorporating capacitance, resistance, drive strength, timing information, and circuit type [3]. The analysis is broken down into the two clock phases and the results are compared to predetermined limits for different circuit configurations. This CAD tool was used to screen out nonfailing circuits as well as to show the changes in coupling to circuits which were being fixed. Most failures were fixed with layout edits to increase the spacing from aggressor nodes and decrease the spacing to helpful nodes (such as VDD/VSS). Any coupling failure not fixed in layout was extensively analyzed with SPICE to ensure correct operation.

VI. Conclusion

The introduction of MVI to the Alpha instruction set brought with it many interesting circuit issues. Several technical and time-to-market tradeoffs had to be made to ensure a functional implementation in a timely fashion while still supplying the necessary performance to drive high speed motion video algorithms. Functional parts have booted the Windows NT[1] operating system on first pass silicon at frequencies up to 550 MHz.

[1] Windows NT is a trademark of Microsoft Corporation.

Acknowledgment

The authors acknowledge the contributions of the following individuals: A. Jain, R. Preston, P. Bannon, M. Bertone, R. Blake-Campos, G. Bouchard, D. Brasili, K. Clark, S. Kobayashi, B. Lilly, S. Mehta, B. Miller, A. Olesin, Y. Saito, and V. Yalala.

References

[1] P. Bannon, "Enhancing motion video performance in the Alpha architecture," presented at Microprocessor Forum 96, Oct. 1996.
[2] W. Bowhill *et al.*, "A 300 MHz 64 b quad-issue CMOS microprocessor," in *ISSCC Dig. Tech. Papers*, Feb. 1995, pp. 182–183.
[3] P. Gronowski *et al.*, "A 433 MHz 64-b quad issue RISC microprocessor," *IEEE J. Solid-State Circuits*, vol. 31, pp. 1687–1696, Nov. 1996.
[4] A. Jain *et al.*, "A 1.38 cm^2 550 MHz microprocessor with multi-media extensions," in *ISSCC Dig. Tech. Papers*, Feb. 1997, pp. 174–175.
[5] O. Lempel *et al.*, "Intel's MMX technology—A new instruction set extension," in *Compcon Spring'97*, Feb. 23–26, 1997.
[6] R. Sites, Ed., *Alpha Reference Manual.* Burlington, MA: Digital, 1992.
[7] M. Tremblay *et al.*, "The visual instruction set (VIS) in ultraSparc," presented at Compcon Spring'95, Mar. 5–9, 1995.



David A. Carlson received the B.S.E.E. degree in 1993 and the M.S.E.E. degree in 1994, both from the University of Texas at Austin.

On the 21164PC, he was responsible for the integer unit. In addition to implementation responsibilities, he has also designed several CAD tools to aid electrical and functional verification. He is currently working on high-performance arithmetic for a new Alpha microprocessor at Digital Equipment, Hudson, MA.



Ruben W. Castelino received the B.S.E.E. degree from the University of Cincinnati, OH, in 1988.

He is a Senior Hardware Engineer in Digital Semiconductor, Hudson, MA. He has contributed to the design of several high-performance microprocessors in the past nine years. In his work on the Alpha 21164PC, he was responsible for the bus interface unit. He is currently responsible for the design and implementation of the pad ring for a new Alpha microprocessor. He holds one patent.



Robert O. Mueller received the B.S. degree in computer and systems engineering from Rensselaer Polytechnic Institute, Troy, NY.

After graduation, he joined Digital Semiconductor in 1990. He started working on one of the last VAX generation chips, but quickly moved on to the Alpha CPU design group where he has spent at least five years designing leading edge microprocessors. In his work on the Alpha 21164PC, he contributed to the design and implementation of the Memory Management Unit. As a Senior Hardware Engineer in Digital Semiconductor, Hudson, MA, he is currently involved in the implementation of the instruction issue unit for a new Alpha microprocessor.

Authorized licensed use limited to: University of Sydney. Downloaded on November 03,2026 at 22:54:07 UTC from IEEE Xplore. Restrictions apply.